Correction complète du questionnaire

Questions 1 à 15

Table des matières

1 Question 1 1

2 Question 2 1

3 Question 3 2

4 Question 4 2

5 Question 5 2

6 Question 6 2

7 Question 7 2

8 Question 8 2

9 Question 9 3

10 Question 10 3

11 Question 11 3

12 Question 12 4

13 Question 13 4

14 Question 14 4

15 Question 15 5

1 Question 1

Votre réponse : *« Un type de données mutable ne peut pas être modifiable. . . »*
Incorrect : c'est l'inverse.

Correction : Un type de données **mutable** est un type dont le contenu **peut être modifié après sa création**, sans que l'objet ne change d'identité (l'adresse mémoire reste la même). Par exemple, on peut ajouter, supprimer ou modifier des éléments d'une liste directement.

2 Question 2

Votre réponse : `String, Tuple`
Correct. Les chaînes de caractères (`str`) et les tuples (`tuple`) sont immuables en Python.

3 Question 3

Votre réponse : *« Une fonction est comme une section dans un code qui prend des arguments et retourne un résultat dépendant de la logique dans la fonction. »*

Correct. Le concept de boîte noire signifie qu'on utilise la fonction par son interface (entrées → sorties), sans avoir à connaître son implémentation interne.

4 Question 4

Votre réponse : `assert somme(2, 3) == 5`

Correct. C'est un test unitaire via une assertion. On vérifie le comportement observable (entrée/sortie) sans regarder le code interne.

5 Question 5

Votre réponse : *« On n'a pas besoin de les implémenter, juste importer dans son code et l'utiliser. »*

Correct. Les modules permettent la réutilisation de code déjà écrit et testé.

6 Question 6

Votre réponse : Explication correcte avec un exemple utilisant `if __name__ == "__main__":`.

Correct. La variable `__name__` vaut `"__main__"` quand le fichier est exécuté directement, et vaut le nom du module quand il est importé.

7 Question 7

Code :

```
compteur = 0
while a >= -1 and a <= 1:
    a = a + 1
    compteur = compteur + 1
print(compteur)
```

Réponse : 3.

8 Question 8

Justification :

La boucle s'exécute tant que `a` est entre -1 et 1 inclus. Pour maximiser `compteur`, il faut maximiser le nombre d'itérations, donc choisir la plus petite valeur possible : $a = -1$.

Itération	`a` (avant)	`a` (après)	`compteur`
1	-1	0	1
2	0	1	2
3	1	2	3

À la quatrième vérification, `a = 2` ne satisfait plus `a <= 1` : la boucle s'arrête. La valeur maximale affichée est 3.

9 Question 9

Instruction :

```
test = b == "neuf" or abs(a) % 5 > 1
```

Étapes d'exécution (dans l'ordre) :

1. **Évaluer** `b == "neuf"` : l'interpréteur compare la valeur de `b` à la chaîne `"neuf"`. Le résultat est `True` ou `False`.
2. **Court-circuit du** `or` **:**
   - Si `b == "neuf"` est `True`, l'expression de droite (`abs(a) % 5 > 1`) n'est pas évaluée. Le résultat de l'expression `or` est `True`.
   - Si `b == "neuf"` est `False`, l'interpréteur doit évaluer la partie droite.
3. **Si l'évaluation de la partie droite est nécessaire :**
   a. Appeler la fonction `abs(a)` pour obtenir la valeur absolue de `a`.
   b. Calculer `abs(a) % 5` (le reste de la division entière par 5).
   c. Comparer ce résultat à 1 avec l'opérateur `>` : le résultat est `True` ou `False`.
4. **Évaluer l'opérateur** `or` **:** il suffit qu'au moins une des deux expressions soit vraie pour que le résultat soit vrai.
5. **Assignation :** le résultat final (booléen) est stocké dans la variable `test`.

10 Question 10

Code :

```
def ma_fonction(liste_1, liste_2):
    verdict = len(liste_1) == len(liste_2)
    i = 0
    while i < len(liste_1):
        if liste_1[i] in liste_2:
            verdict = False
        i = i + 1
    return verdict
```

Explication :

La fonction `ma_fonction` vérifie si deux listes sont de **même taille** et **n'ont aucun élément en commun** (elles sont disjointes). Elle retourne `True` si les deux conditions sont respectées, et `False` sinon. Si les listes n'ont pas la même longueur, ou si au moins un élément de `liste_1` se trouve dans `liste_2`, le verdict devient `False`.

11 Question 11

Code :

```
from module_magique import fonction_obscure

def fonction_mysterieuse(une_liste):
    d = {}
    for element in une_liste:
        d[element] = fonction_obscure(element)
    return d
```

Explication :

La fonction `fonction_mysterieuse` construit et retourne un **dictionnaire** qui associe chaque élément de la liste à une valeur calculée par `fonction_obscure`. Elle parcourt la liste et, pour chaque élément, crée une entrée dans le dictionnaire où la clé est l'élément et la valeur est le résultat de `fonction_obscure(element)`. Si un élément apparaît plusieurs fois, seule la dernière valeur calculée sera conservée (les clés d'un dictionnaire sont uniques).

12 Question 12

Observation :

```
>>> print(fonction_mysterieuse(["je", "pense", "donc", "je", "suis"]))
{'je': 2, 'pense': 4, 'donc': 4, 'suis': 3}
```

Analyse :

- `"je"` → 2 : 2 caractères uniques (`j`, `e`).
- `"pense"` → 4 : 4 caractères uniques (`p`, `e`, `n`, `s`).
- `"donc"` → 4 : 4 caractères uniques (`d`, `o`, `n`, `c`).
- `"suis"` → 3 : 3 caractères uniques (`s`, `u`, `i`).

Réponse : La fonction `fonction_obscure` retourne le **nombre de caractères distincts (uniques)** dans une chaîne de caractères.

13 Question 13

```python
def afficher_index_et_valeur_max(liste):
    if len(liste) == 0:
        return

    index_max = 0
    valeur_max = liste[0]

    for i in range(1, len(liste)):
        if liste[i] > valeur_max:
            valeur_max = liste[i]
            index_max = i

    print("L'index de la valeur maximale est:", index_max)
    print("La valeur maximale est:", valeur_max)
```

14 Question 14

```python
def chercher_remplacer(chaine, caractere_a_remplacer, caractere_remplacement,
    index_debut, index_fin):
    nouvelle_chaine = ""

    for i in range(len(chaine)):
        if index_debut <= i <= index_fin and chaine[i] == caractere_a_remplacer:
            nouvelle_chaine += caractere_remplacement
        else:
            nouvelle_chaine += chaine[i]

    return nouvelle_chaine
```

Vérification avec l'exemple :

```
>>> ma_chaine = "Ceci est une chane de caractres."
>>> chercher_remplacer(ma_chaine, " ", "*", 6, 21)
"Ceci est*une*chane*de caractres."
```

Correct. Les espaces aux index 6, 11 et 16 sont remplacés par `*`, les autres restent inchangés.

15 Question 15

```
# Test 1 : Cas standard - remplacement dans la plage
assert chercher_remplacer("Ceci est une chane de caractres.", " ", "*", 6, 21) == "
    Ceci est*une*chane*de caractres."

# Test 2 : Aucun caractre cible dans la plage : chane inchange
assert chercher_remplacer("bonjour", "x", "y", 0, 6) == "bonjour"

# Test 3 : Remplacement aux bornes exactes (dbut et fin inclus)
assert chercher_remplacer("abcabc", "a", "X", 0, 3) == "XbcXbc"
```